HW1 实验报告

Logic Synthesis

Tongbin Zhao 学号：2300012831 2026 年 10 月 8 日

Problem 1：布尔差分

Problem 1.

证明：为书写简便，记：$f_0 = f(x=0,u)$，$f_1 = f(x=1,u)$

$g_0 = g(x=0,u)$，$g_1 = g(x=1,u)$.

因此：$D_x f = f_0 \oplus f_1$，$D_x g = g_0 \oplus g_1$

由于原式左边为与 x 无关的函数，右边为

与 x 有关的函数，考虑枚举 x 的取值分类证明

① 当 $x=0$ 时：

右边 $= f_0(g_0 \oplus g_1) \oplus g_0(f_0 \oplus f_1) \oplus (f_0 \oplus f_1)(g_0 \oplus g_1)$

由于：$a(b \oplus c) = a\bar{b}c + ab\bar{c} = ab \oplus ac$

因此：右边 $= f_0 g_0 \oplus f_0 g_1 \oplus g_0(f_0 \oplus f_1) \oplus g_0(f_0 \oplus f_1) \oplus$

$g_1(f_0 \oplus f_1)$.

由于：$a \oplus a = 0$，$0 \oplus a = a$. 故：

右边 $= f_0 g_0 \oplus f_0 g_1 \oplus f_0 g_1 \oplus f_1 g_1 = f_0 g_0 \oplus f_1 g_1$

而左边 $= D_x(fg) = f_0 g_0 \oplus f_1 g_1$，

因此 $x=0$ 时，原式成立，

手写解答 · 原稿第 1 页（续见下一页）

Problem 1（续）

②当 $x_1=1$ 时：左边化为 $f g_0 \oplus f g_1$

右边 $= f_1(g_0 \oplus g_1) \oplus g_1(f_0 \oplus f_1) \oplus g_0(f_0 \oplus f_1) \oplus g_1(f_0 \oplus f_1)$

$= f g_0 \oplus f g_1 \oplus g_0 f_0 \oplus g_0 f_1 = f g_0 \oplus f g_1 =$ 左边

综上，原式成立，证毕.

手写解答 · 原稿第 2 页上半部

Problem 2：工艺映射与树覆盖

Problem 2

(a). 证明：布尔函数有 $\wedge$(AND), $\vee$(OR), $\neg$(NOT) 三种.

AND: $a \wedge b = \neg(\neg(a \wedge b)) = \neg a \uparrow b$.

OR: $a \vee b = \neg(\neg(a \vee b)) = \neg[(\neg a) \wedge (\neg b)]$

$= (\neg a) \uparrow (\neg b)$

NOT: $\neg a = \neg(a \wedge a) = a \uparrow a$.

因此 NAND 是完备的，证毕.

(b) $f(a,b,c,d) = (a \wedge d) \vee (b \wedge c \wedge d) \vee (a \wedge c)$

$= \neg\left(a \uparrow d \wedge \neg(b \wedge c \wedge d) \wedge a \uparrow c\right)$

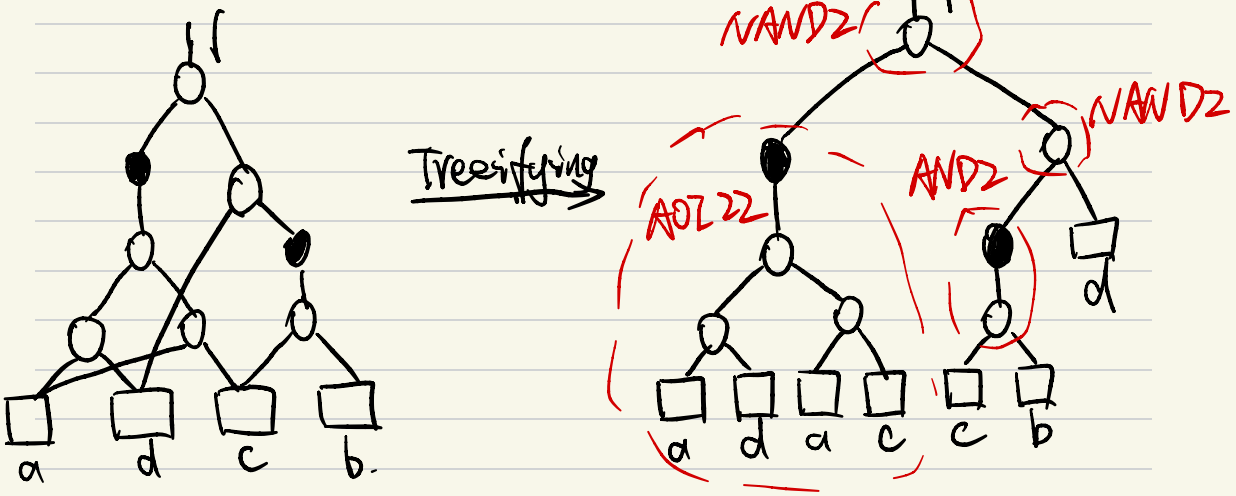
手写解答 · 原稿第 2 页下半部

Problem 2（续）

$$= \neg(a\uparrow d \wedge (b \wedge c)\uparrow d \wedge a\uparrow c)$$

$$= \{\neg[(a\uparrow d)\uparrow(a\uparrow c)]\}\uparrow[(\neg(b\uparrow c))\uparrow d]$$

(c) f对应的AIG为：



红色标注即最佳tree covering

共需 2个NAND2，1个AND2，1个AOI22

Problem 3：Tree Covering 算法

(a) 关于 Tree Covering 算法的讨论

(i) **树上的最优性**。在 Problem 2(c) 中，主输入（PI）节点是免费的，而该例的树化（treeifying）可以通过复制输入节点完成，因此没有额外面积开销。在一棵树上，各子树的最优解不受其上层节点和其他子树映射方案的影响，因此可以通过动态规划（DP）得到最优的树覆盖。

(ii) **DAG 上的最优性丢失**。在 DAG 上直接运行 tree covering 算法不保证得到全局最优解。一棵子树根节点匹配到的 pattern tree，可能覆盖其他分支也会使用的节点。因而各子问题之间存在共享，映射问题不再完全解耦，独立求解各子问题的 DP 无法保证全局最优。

(iii) **同时跟踪两种极性的作用**。有些库门的 pattern tree 自带反相边，其面积小于先将反相边全部用 `inv1` 实现、再对剩余节点进行 tree covering 的总面积。如果对每条反相边都固定插入一个 `inv1`，就可能错过更便宜的局部覆盖方案。

(b)-(d) 算法设计

此部分为 Tree Covering 算法代码的实现任务，代码在 `src` 文件夹中，根据 `README.md` 文件中的说明可以完整复现测试结果。

附录：AI 使用记录

本次作业使用 AI 辅助完成了以下工作：

- 检查 Problem 3 的两个函数，帮助笔者进行 debug 并使用附带检查器及 ABC 完成四个测试电路的评估。
- 讨论 tree covering 在 DAG 上的适用方式，以及共享逻辑对面积最优性的影响。
- 将本人手写的 Problem 1、Problem 2 解答嵌入报告，整理 LaTeX 排版并编译导出 PDF。